COSC 312
HW 8b: Turing Machine Project
19 April 2026

Territorial Battle Simulation

Contributors

Vincent Guo - creating duplicator, translator, unary counter, and drafting writeup.
Minjae Bae - idea execution, creating parser, debugging

Problem Description

The goal of this project was to develop a Turing Machine that could simulate a territorial battle between two nanorobot swarms in a crowded battlefield (a 1D tape), where the user can initalize the starting board configuration, some rules, and the number of iterations.

The Turing Machine then iteratively applies the rules, scanning from left to right, applying the user-defined rules on each nanorobot.

Starting the Machine

The program takes input in the following format:
`[.|.|.$.|.|.*N]S#`

- `.` and `S` satisfy `(A|B)*`, *no spaces.*
- `N` is a sequence of 1's which tells the program how many time to iterate. `*111` would mean iterate 3 times. `*11111` would mean iterate 5 times. If `N` is not set, the program simply exits. `*` must be part of the input.
- `S` **MUST** be `#`-terminated, with only a single `#`. `#` is used to mark the end of the board. Without `#`, expect infinite loops.
- Rule delimiters are `[`, `|`, `$`, `*`.

Outputs are then spat out in the same `[.|.|.$.|.|.*N]S#` format as well. If input strings are valid, the program will enter an accept state.
If input strings are invalid, expect undefined behavior. We assume inputs are valid and do not do any input validation.

We will refer to the `[.|.|.$.|.|.*]` segment as the **RULES** and the `S#` segment as the **BOARD**.

NOTE: you may have to scroll right a bit to see the output, though STEM usually does a good job of keeping up with the tape.

Example Inputs:

GOOD:

- `[||$||*1]#`
  Expected Output: `[||$||*]#`

- `[B||$A||*1]ABABABABAB#`
  Expected Output: `[B||$A||*]#`
- `[AA|AA|AB$A|BB|A*1]BBAAABA#`
  Expected Output: `[AA|AA|AB$A|BB|A*]AABAB#`
- `[ABABABAB|AAAA|BBBBB$A|BB|*1]ABABABABA#`
  Expected Output: `[ABABABAB|AAAA|BBBBB$A|BB|*]BBBBB#`
- `[A|A|BB$B|B|AA*1]ABABABABA#`
  Expected Output: `[A|A|BB$B|B|AA*]BBAABBAABBAABBAABB#`

BAD: The Turing Machine cannot process these kinds of inputs.

- `[B|AA|$A|BB|*1]BAAABBB` (missing #)
- `[B|AA|$A|B B|*1]BAAABBB` (not (A|B)*)
- `[b|aa|$a|bb|*1]BBBBBBB#` (invalid characters)
- `[B|AA$A|BB|*1]BBBBBBB#` (missing delimiters)
- `[B|AA|$A|BB|]BBBBBBB#` (missing *)
- `[B|AA|$A|BB|*999]BBBBBBB#` (invalid number - must be in unary)

The Turing Machine should work for arbitrarily long strings, but it may take a couple of minutes. Also, keep N low (<`1111`), as things get out of hand really quickly. STEM may freeze if the rules encourage growth too much and N is set to a large number.

Understanding the Rules

`[.|.|.$.|.|.*N]` is where the user defines the rules for the simulation.

The rules for species `A` are defined in the first `[.|.|.$` chunk.
The rules for species `B` are defined in the second `$.|.|.*` chunk.

Species rules are interpreted as **HUNT|REPRODUCE|TRANSFORM**. That might not make much sense, so let us give an example.

E.g. `[AA|AA|AB$A|BB|*1]BBAAABA#`
First, we look at the rules for species `A` in the `[BB|A|AB$` chunk.

Interpreting `[BB|A|AB$`:

1. `[AA|`: this defines what chunk `A` can hunt. If `A` is adjacent to a sequence of **AT LEAST** 2 `A`'s, the 2 adjacent `A`'s are marked as prey.
2. `|AA|`: if `A` **meets** the hunting condition, **REPLACE** `A` with `AA`. **Note that if the hunting rule is empty, `A` will always execute this path.**
3. `|AB$`: if `A` **fails to meet** the hunting condition, **REPLACE** `A` with `AB`.

Order matters. Specifying that species `A` can hunt `BA` does not allow them to hunt `AB`.
E.g. if the board were like this and we were interested in the lone `A`, which we say can hunt `BA`

- `BA A` would lead to a successful hunt.
- `A BA` would lead to a successful hunt.

- `A AB` would **NOT** lead to a successful hunt.
- `AB A` would **NOT** lead to a successful hunt.

if the hunting condition is blank, then the hunt is always successful.

Interpreting `$A|BB|*`:

1. `$A|`: this defines what chunk `B` can hunt. If `B` is adjacent to a sequence of **AT LEAST** 1 `A`, the adjacent `A`'s is marked as prey.
2. `|BB|`: if `B` **meets** the hunting condition, **REPLACE** `B` with `BB`.
3. `|*`: if `B` **fails to meet** the hunting condition, **REPLACE** `B` with nothing.

Note that if 2 delimiters have no `A`'s or `B`'s in between, e.g. `[|`, `||`, `|$`, `$|`, `|*`, it is assumed that the rule is empty.

Parsing Precedence

The parser always evaluates left precedence over right precedence. An swarm only attempts to hunt other swarms to the right if it cannot find the nanorobots it needs to hunt on the left.

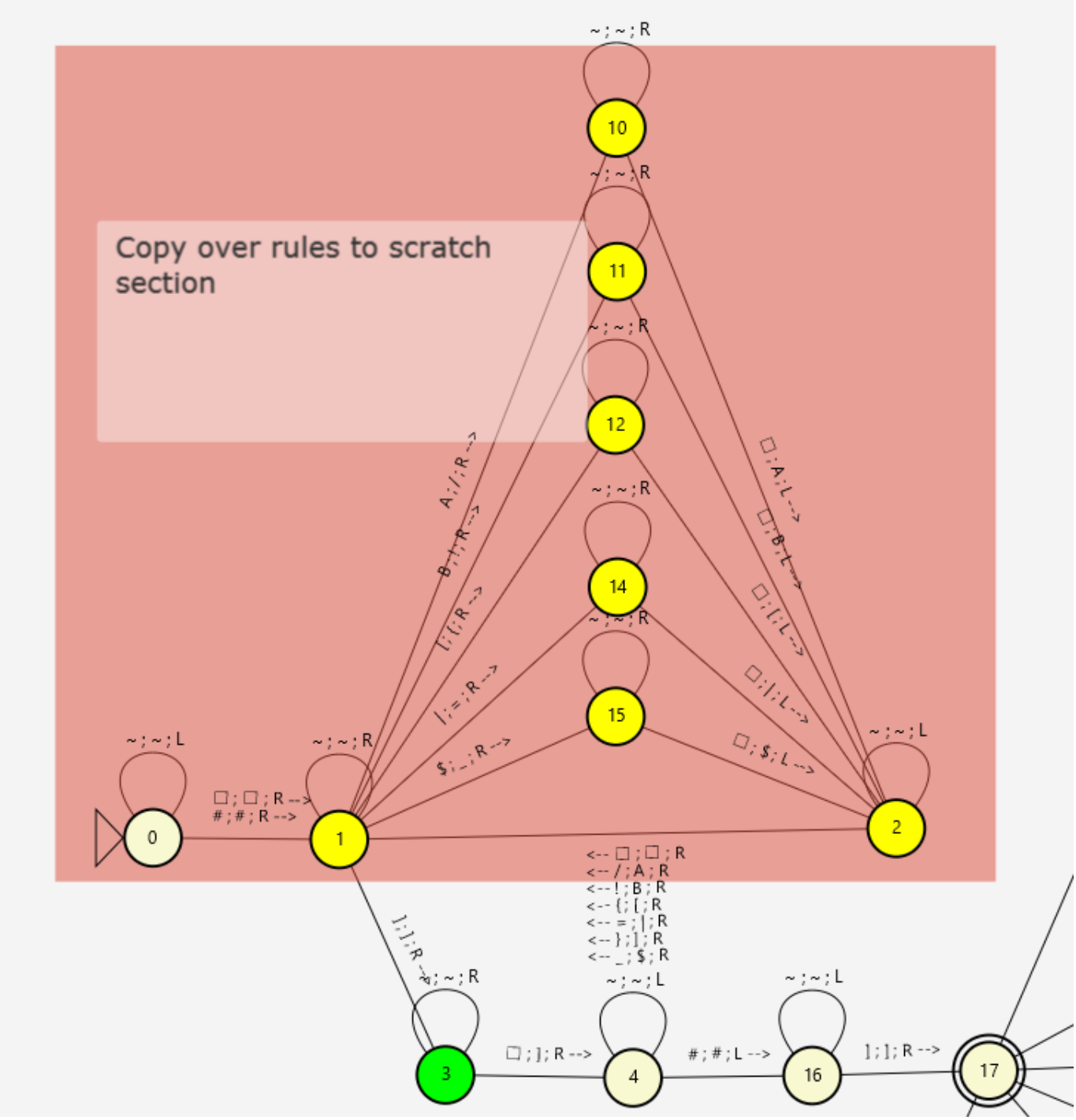
*If we were currently reading the `A` using the same rules, `BBABB` evaluates to `AB BB` **not** `BB ABB` (spaces added for clarity) after hunting. This prevents ambiguity.*

Also, nanorbbots that have already been marked as prey by one nanorobot cannot be marked as prey again by another nanorobot. Nanorobots that likewise have sucessfully hunted others cannot be hunted by another swarm (these nanorobots have already established themselves as capable fighters, and it would make little sense to hunt them for that epoch). Everyone else? Fair game. This will make more sense in the example.

Approach

Step 1: Copying Rules

Before parsing the board, we copy the RULES section to the end. We do this to facilitate recursive generations of the board. This is done via a duplicator.



Node 0 sets the turing pointer to the first element of the input string. We then loop (yellow nodes), marking elements as we go and copying characters to the first instance of whitespace, before looping back and using the mark to find the next element we need to copy. This is done until we reach the end of the rules, marked by `]`. We also use a unary counter to keep track of how many generations left to simulate.

After this step:

- `[AA|AA|AB$A|BB|*1]BBAAABA#` becomes `[AA|AA|AB$A|BB|*-]BBAAABA#[AA|AA|AB$A|BB|*]`.
- `[||$||*1]BBABBA#` becomes `[||$||*-]BBABBA#[||$||*]`.

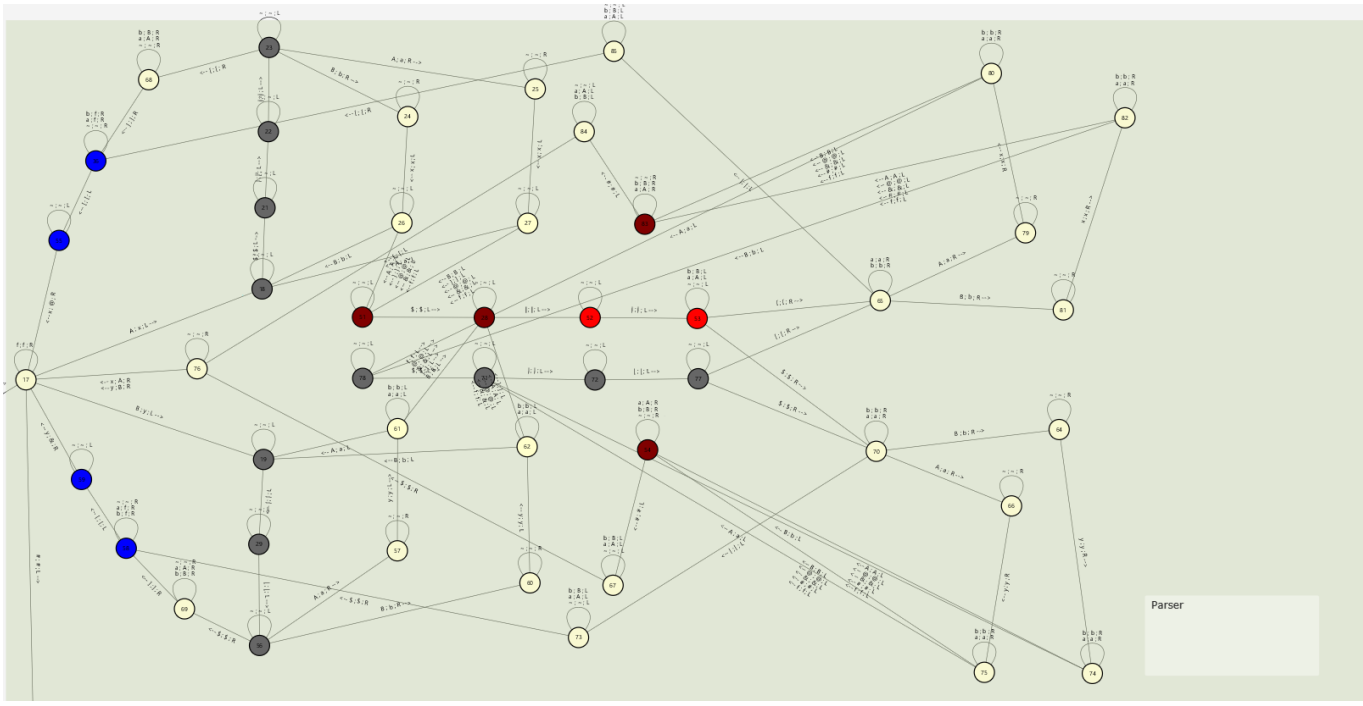
Step 2: Parsing the board

Continuing with `[AA|AA|AB$A|BB|*1]BBAAABA#`.

(`[AA|AA|AB$A|BB|*-]BBAAABA#[AA|AA|AB$A|BB|*]` after step 1)

We read the board `BBAAABA#`.

How is this done?
Through the left and right parsers. Here's an image:



The entry point is on node 17.

- Blue Nodes - these encode successful matching.
- Magenta Nodes - these encode mismatches for one singular check. If we are using the left parser, we move to the right parser. Else the nanorobot fails to hunt.
- Bright Red Nodes - these encode where exactly the left parser switches to the right parser.
- Gray Nodes - fetches the ruleset

The rest of the nodes mostly deal with intermediate steps (e.g. we just read an A from the rules).

The parser exits through node 17 as well upon reading a `#` symbol, which then transitions to the next segment through node 110.

The parser is complicated, so to explain it, let us scan through a string like we are the parser.

Parser walkthrough

We use the input string `[AA|AA|AB$A|BB|*1]BBAAABA#`

There is a `1` after the `*`, so we should not exit early.
After the duplicator does its copying, we have `[AA|AA|AB$A|BB|*-]BBAAABA#[AA|AA|AB$A|BB|*]` entering into the parser.

(i):

We read `B`. This means use `B`'s rules (`$A|BB|]`).

We first keep track of our current char by marking the robot we are currently interested in (`B`) as `y`, and then scan left to the first letter or `$` we see. We have to ping-pong back and forth because we do not know how long the sequence of what swarm `B` can hunt is.

We see `B` can hunt `A`. We then scan back to `y` and read `[`.

Since `B`'s can only hunt `A`, `B` does not meet the requirements to hunt and then reproduce.

(current state: `[AA|AA|AB$A|BB|*-]yBAAABA#[AA|AA|AB$A|BB|*]`)

Since `y`'s hunt was unsuccessful, we scan right. We see that to the right of `y` is a `B`. But `B` can only hunt `A`, not `B`. We revert `y` back to `B`, indicating that the swarm could not hunt for food. Then, we start the scanning process over again for the next non-lowercase element.

(ii):

The next element we scan is also a `B`, so it gets converted temporarily to a `y`.

(current state: `[AA|AA|AB$A|BB|*-]ByAAABA#[AA|AA|AB$A|BB|*]`)

We now repeat **(i)**. Scanning left, we find a `B`. That does not match an `A`, so we continue by scanning right.

Scanning right, we find `A`. That matches what `B` can hunt. However, we do not know if this matches the full chunk. Thus we must denote it with lowercase `a` as viable prey, and check for the next part of the chunk in the rules.

In this case, we read `|`, thus we mark `A` as `B`'s prey by writing `a` as `f` and continue to the next element. `B` also gets converted to an `&` to signify that it will reproduce.

(current state: `[AA|AA|AB$A|BB|*-]B&fAABA#[AA|AA|AB$A|BB|*]`)

(iii):

We scan `f` next. But the `f` notation means that it has already been marked as prey. So we skip `f` and scan the next character (the `A`). Mark `A` as `x` to denote that it is the current nanorobot we are interested in.

(current state: `[AA|AA|AB$A|BB|*-]B&fxABA#[AA|AA|AB$A|BB|*]`)

To the left of `x` is an `f`. However, it is a `f`, so it has already been marked as prey by another swarm. `x` cannot feast on it.

We then scan right. To the right of `x` is `AB` not `AA`, so `x` cannot hunt. We revert `x` back to `A`, signifying it did not successfully hunt and can still be eaten. We continue.

(iv):

You should get the hang of this by now. Replace the current scanned nanorobot with a temporary variable (using `x` for `A` and `y` for `B`), then scan left and right.

(current state: `[AA|AA|AB$A|BB|*-]B&fAxBA#[AA|AA|AB$A|BB|*]`)

To the left, there is `fA`, but `f` doesn't count, since that food source has already been taken by someone else.

Scanning to the right, we see `BA`. That doesn't match `AAA`, so our current marked swarm gets to transform. Our `x` returns to being an `A`.

(v)

Again, replace the current scanned variable with a fitting alias. We use `y` since this is an swarm of species `B`.

(current state: `[AA|AA|AB$A|BB|*-]B&fAAyA#[AA|AA|AB$A|BB|*]`)

We scan left first. To the left of `y` is `AA`, so our current swarm has found his prey.

We modify the `A` left of the `y` to `a`, to show that the `A` has been preyed upon.

Since `y` had a nice meal, it can now reproduce, so we replace `y` with `&` and then move on to the final swarm. Note that since the swarm got its food from hunting creatures to the left of it, it sees no need to hunt swarm to the right of it.

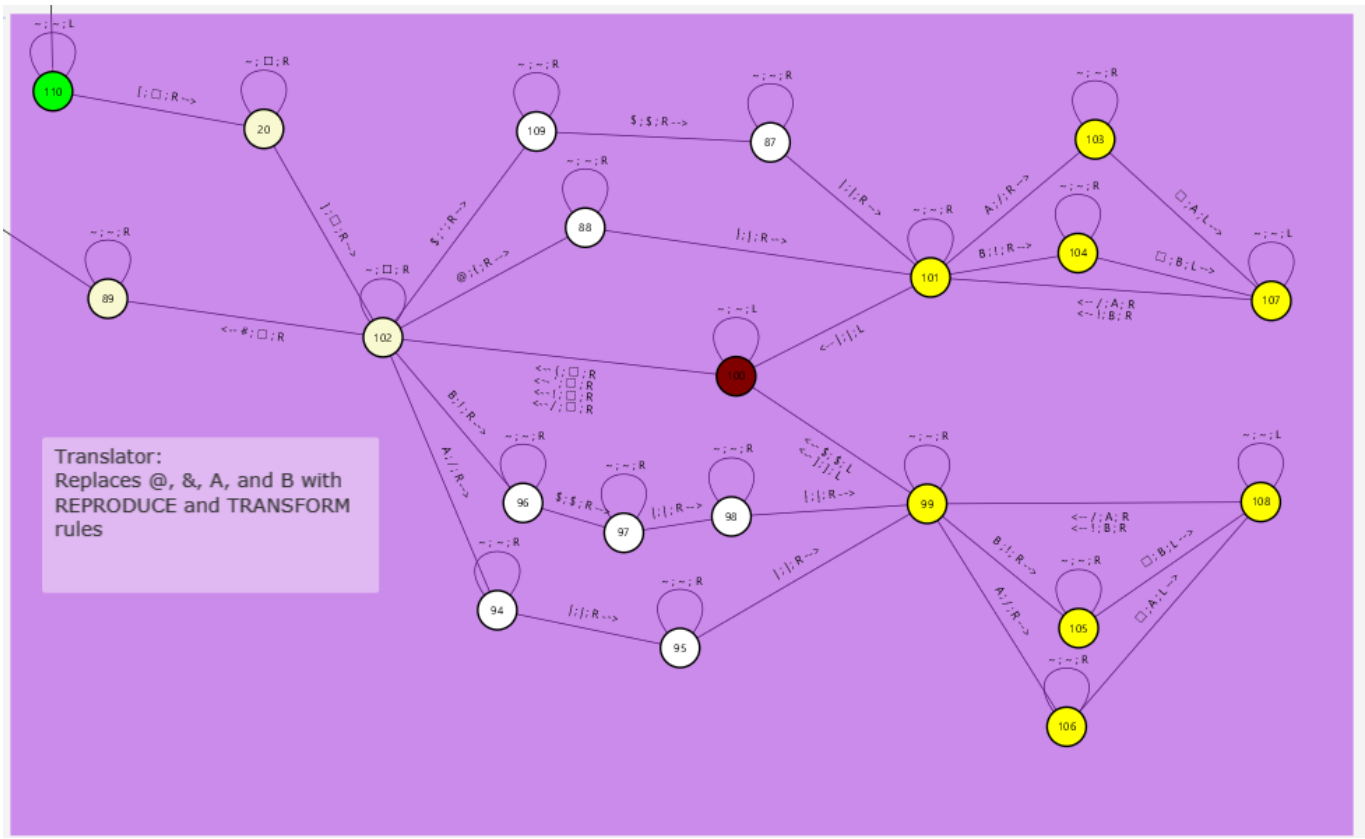
(current state: `[AA|AA|AB$A|BB|*-]B&fAf&A#[AA|AA|AB$A|BB|*]`)

(vi)

Now we look at the `A` right before the `#`. To the left of it is an `&`, meaning an swarm who has already hunted.

To the right is `#`. That means the end of the board. Since neither scans bring up `AA`, the last nanorobot we see does not successfully hunt, so it transforms.

Step 3: Translation & Deletion



- Green node: entry point
- White node: transition path for given element
- Yellow node: scan and copy letter by letter (same idea as the duplicator)
- Maroon node: temporary error state

So now we have a string like this: `[AA|AA|AB$A|BB|*-]B&fAf&A#[AA|AA|AB$A|BB|*]`

We want our final output to be like so: `[.|.|.$.|.|.]S#`

That way, the output of this program can be cycled over and over again.

The last thing to do is to copy over the new board so we get another string in the following format (`[.|.|.$.|.|.]S#`) without the `@` and `&`.

This is done via a very simple copy algorithm, applied onto the start of the board. Not the rules, since those were copied earlier:

- Capital letters (`A`, `B`) are swarm who were unable to hunt but were lucky enough to not be preyed upon. This means copying over the corresponding TRANSFORM rules when one of these is encountered.
- `@` and `&` are used for encoding swarm for species `A` and `B` respectively who were able to hunt.
- Lowercase letters (`f`) are the unlucky ones. They were preyed upon and are now presumed to be dead. We will not keep records of them in the next board.

Trace:

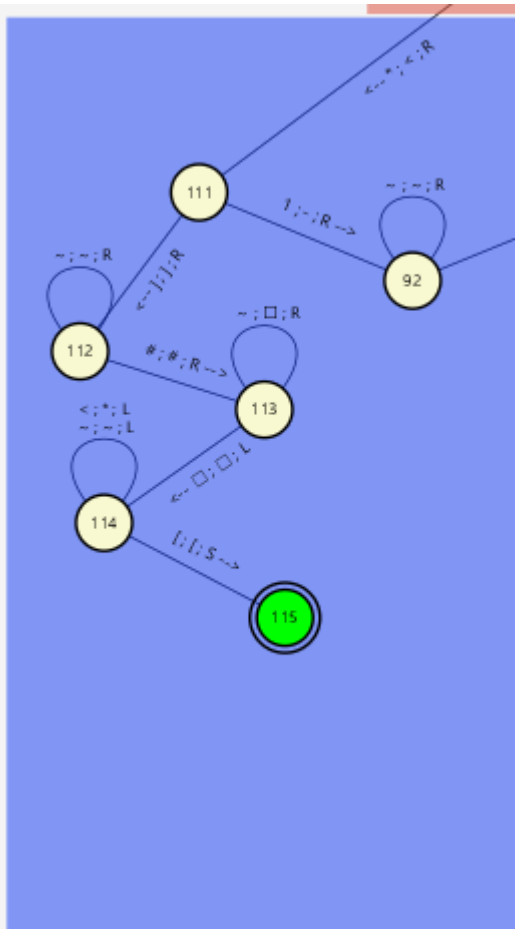
- `B &fAf&A` (B->nothing)
- `& fAf&A` (&->BB)
- `BB f Af&A` (f->nothing, presumed to be dead)
- `BB A f&A` (A->AB)

- `BBAB f &A` (f->nothing, presumed to be dead)
- `BBAB & A` (&->BB)
- `BBABBB A` (A->AB)
- `BBABBBAB` (Complete board)
- `BBABBBAB#` (Add #-termination)

When done to completion, `[AA|AA|AB$A|BB|*-]B&aAa&A#[AA|AA|AB$A|BB|*]` becomes `[AA|AA|AB$A|BB|*]BBABBBAB#` which is now a valid input string!

This marks the end of one cycle. The output from here is fed back to the start state:

Step 4: Iteration



Not much to say here. Basically, reading a `*` from the duplicator leads to state 111, which branches based on the next char. If the char right after `*` is a `1`, denote the `1` as `-` (this doesn't get copied by subsequent duplications) and restart the program with newly updated board.

Else if we read a `]`, clean up a partial duplication and terminate the program.

In our case of `[AA|AA|AB$A|BB|*]BBABBBAB#`, since `*`'s next character is `]`, the program halts here.